UNIVERSITY OF MSILA
COMPUTER SCIENCE DEPARTMENT

L2-INF/INF-D
Object-oriented programming I
Problem set 1: Procedural Programming and Its Limitations

04/10/2026
two weeks

Exercise 1: Limits of procedural programming

Consider the following C program, which manages a small array of students (grades are marks out of 20).

```c
#include <stdio.h>
#include <string.h>
#define NB_STUDENTS 4
#define NAME_LEN 10
typedef struct {
  char name[NAME_LEN];
  int age;
  float grade;
} Student;

void init_student(Student *s, char *name, int age, float grade) {
  strncpy(s->name, name, NAME_LEN - 1);
  s->name[NAME_LEN - 1] = '\0';
  s->age = age;
  s->grade = grade;
}

void print_student(Student *s) {
  printf("%-30s | age: %3d | grade: %5.2f\n", s->name, s->age, s->grade);
}

void print_all(Student tab[], int n) {
  for (int i = 0; i < n; i++)
    print_student(&tab[i]);
}

float average_grade(Student tab[], int n) {
  float sum = 0.0f;
  for (int i = 0; i < n; i++)
    sum += tab[i].grade;
  return (n > 0) ? sum / n : 0.0f;
}

int main(void) {
  Student students[NB_STUDENTS];
  init_student(&students[0], "Ali", 18, 24);
  init_student(&students[1], "Omar", 19, 7.75);
  init_student(&students[2], "Amira", 18, 13);
  init_student(&students[3], "Ahlem", -20, 8.5);
  print_all(students, NB_STUDENTS);
  printf("\nAverage grade: %.2f\n", average_grade(students, NB_STUDENTS));
  return 0;
}
```

Part 1: Observing the Program

1. Compile and run the program.
2. Examine the output. What is wrong with Ahlem's age and Ali's grade?
3. What type of error do these invalid values represent?

Part 2: Implementing Data Validation

4. Modify `init_student()` so that it checks the validity of the student's data. If the data is invalid, the program must exit with a message indicating the error (you may need `#include <stdlib.h>`):
   - the age must be between 1 and 100;
   - the grade must be between 0 and 20.

```c
if (age < 1 || age > 100) {
  printf("wrong age %d for student: %s\n", age, name);
  exit(-1);
}
if (grade < 0 || grade > 20) {
  printf("wrong grade %.2f for student: %s\n", grade, name);
  exit(-1);
}
```

5. Run the program again. Correct the invalid values in `main()` one by one until the program runs to completion.

Part 3: The Limits of Validation

In `main()`, just before the call to `print_all()`, add:

```c
students[0].grade = 57.5f;
students[1].age = -3;
strcpy(students[2].name, "Boubakri Taki eddin");
```

6. Run the program again and verify the results. What do you notice?

7. Does anything in the `Student` structure prevent a programmer from writing the following?

```
students[0].grade = 100;
students[0].age = -50;
```

8. We updated `init_student()` to check the data. Why can't it help in this case?
9. The `strcpy()` call only changes Amira's name. Why did her age and the class average change as well?
10. What type of error is this?

Exercise 2: Moving to C++

Exercise 1 showed the limits of procedural programming. Before addressing them, we first migrate the program to C++ and use a few features that make C code cleaner and safer. Copy your validated program from Exercise 1 (without the lines added in Part 3) into a new `exo2.cpp` and compile it

Part 1: Constants and Types

1. Replace the two `#define` constants with typed constants, e.g. `constexpr int NB_STUDENTS = 4;`. Give two advantages of a typed constant over a macro.
2. In C++, a `struct` name is already a type name. Rewrite the declaration of `Student` without `typedef`.

Part 2: References and `const`

3. Make `init_student()` take a *reference* (`Student &s`) instead of a pointer. Update the body (`s.age` instead of `s->age`) and the calls (no more `&`). Why don't the calls need `&` anymore? Name two differences between a reference and a pointer.
4. Make `print_student()` take a `const Student &s`. Add `s.age = 0;` to its body and compile. What happens, and when is the error detected?
5. Which functions of the program should take `const` parameters, and which should not? Justify your answer.

Part 3: Overloading and Default Arguments

6. Rename both `print_student()` and `print_all()` to `print()`. Why does the program still compile? How does the compiler choose which function to call?
7. Give `init_student()` default values for the age and the grade (`int age = 18, float grade = 0`), then call `init_student(students[2], "Amira", 17);` and `init_student(students[3], "Ahlem");`. Why must default arguments come last in the parameter list?

Exercise 3: From `struct` to `class`

Exercise 2 made the code cleaner, but nothing prevents the errors of Exercise 1, Part 3. Start again from your `exo2.cpp`.

1. Move the `print()` function that prints one student inside `Student` as a member function without parameter, declared `void print() const`, and call it with `tab[i].print()`. Which parameter has disappeared, and how does the function reach the data? What happens if you remove the `const`?
2. Replace `struct` by `class`, keep `print()` public and move the validation into public functions `setAge()`, `setGrade()` and `setName()`; add `getGrade() const` for `average_grade()`, and let `init_student()` call the setters. Then try `students[1].age = -3;` in `main()`. What happens, and why can the validation no longer be bypassed?
3. Which problems of Exercise 1 are now prevented? What can still go wrong?